



Society of Petroleum Engineers

**SPE-229629-MS**

## **Agentic Framework for Intelligent Surrogate Modeling of Simulator-Driven Workflows**

Pradeep Shetty and Antonio Abinader, SLB, Houston, Texas, United States; Prasham Sheth, SLB, Menlo Park, California, United States; Sreekrishnan Ramachandran and Tammy Lam, SLB, Houston, Texas, United States

Copyright 2025, Society of Petroleum Engineers DOI [10.2118/229629-MS](https://doi.org/10.2118/229629-MS)

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

---

### **Abstract**

We introduce a novel large-language-model (LLM) driven agentic framework to automate end-to-end surrogate modeling of simulator-driven workflows in the oil and gas domain. The autonomous agent orchestrates critical steps—initial sampling, simulator execution, adaptive retraining, and strategy switching—to minimize expensive simulator calls while meeting accuracy targets with minimal subject matter expert (SME) intervention. Applied to well-network flow and gas processing simulations, the agent dynamically switches sampling strategies to achieve user-specified accuracy efficiently.

In each iteration, the LLM agent generates candidate input sets, evaluates them via the simulator, and fine-tunes the surrogate model on the expanded dataset. Uncertainty estimates from Monte Carlo (MC) dropout guide an acquisition function that scores candidates (e.g., by residual error or predicted variance) to prioritize informative samples. The agent monitors the surrogate model's error on a validation set and triggers early stopping to prevent overfitting.

Initial results demonstrate that the agent-driven approach can autonomously produce high-fidelity surrogates with intelligent sampling and model updates. The adaptive strategy switching yielded roughly a 6% lower Symmetric Mean Absolute Percentage Error (SMAPE) versus using any single sampling method, and the pipeline converged to the target error threshold with minimal human guidance. This framework is among the first to employ an LLM as an autonomous orchestrator for surrogate modeling workflows, significantly reducing computational cost and SME effort while providing a plug-and-play solution for complex simulator-driven engineering problems.

### **Introduction**

Surrogate models are widely used to approximate complex engineering simulators, enabling faster analyses of oil and gas processes. However, building accurate surrogates traditionally requires significant manual effort in experiment design and tuning. Recent advances in LLM-based agents suggest that AI agents can coordinate complex tasks autonomously (Shen, 2023). In particular, an LLM with advanced reasoning capabilities can serve as a central "brain" to plan experiments, control simulators, and refine models. This paper proposes an **agentic framework** where a single LLM-based agent orchestrates the surrogate

modeling workflow end-to-end. The motivation is to drastically reduce the simulation runs and expert time needed to attain a high-accuracy surrogate. We begin by introducing relevant prior work and then detail the methodology and results of the proposed approach.

## Related Work

LLM-powered agents have emerged for orchestrating tools and models across various domains (Shen, 2023). In the context of engineering and optimization, Xie et al. introduced an LLM-driven system to dynamically configure surrogate models for expensive optimization tasks (Xie, 2025). Wuwu et al. proposed a multi-agent LLM framework to autonomously develop Physics-Informed Neural Network surrogates for solving PDEs (Wuwu, 2025).

Recent surveys have formalized the role of LLMs as autonomous data science agents, capable of managing tasks such as data preprocessing, model training, and evaluation (Wang, 2025). Other frameworks like LaMDAgent demonstrate how LLMs can optimize post-training pipelines, including fine-tuning and model merging (Yano, 2025). In simulation workflows, Liu et al. explored how LLMs can automate experiment design and execution through prompt engineering (Liu, 2025). Scientific LLMs are also gaining traction across disciplines, with Zhang et al. cataloging over 260 models used in scientific discovery (Zhang, 2025).

Active Learning techniques are often employed in surrogate modeling to decide where to sample next; for example, using MC dropout uncertainty estimates (Gal, 2015) or other error indicators to guide sampling. There is also growing interest in using LLMs to discover or improve acquisition functions themselves—for example, the FunBO approach leverages an LLM to generate new Bayesian optimization acquisition functions that generalize across problems (Aglietti, 2024). In the context of physics-informed modeling, recent work has shown how PINNs can be integrated into power system simulations (Nadal, 2025) and geotechnical engineering (Yuan, 2025).

Unlike previous works that focus on specific aspects (surrogate selection, single-model tuning, or new acquisition strategies), our framework integrates an LLM agent to manage the entire surrogate modeling pipeline. This encompasses understanding the simulation context, adaptively sampling data points, selecting among multiple acquisition strategies, and interacting with the simulator and model training in a closed loop.

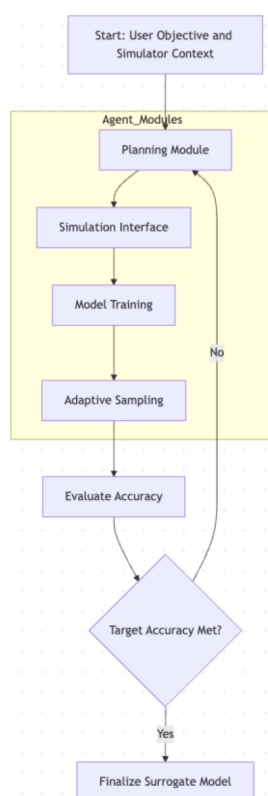
## Methodology

Our methodology section describes the architecture and components of the intelligent surrogate modeling agent, organized into subsections 3.1 through 3.9.

### Architecture of the Autonomous Surrogate Agent

We design an LLM-based agent that plans and controls the surrogate modeling workflow. The agent, implemented with a state-of-the-art LLM, e.g. GPT-4 (OpenAI, 2023), perceives the user's modeling objectives and available tools (simulator, data, etc.), then formulates a step-by-step plan to create the surrogate. The architecture comprises modules for **planning**, **simulation interface**, **model training**, and **analysis**, all coordinated through the LLM's reasoning. The agent's planning module uses prompts and internal logic to determine actions each iteration, such as when to sample new points or retrain the model. Appendix C highlights some of the sample prompts and responses for the agent.

Figure 1 summarizes the architecture of the autonomous agent for surrogate modeling. As shown in Figure 1, the agent begins by interpreting the user's objective and available tools, then iteratively coordinates simulation runs, model updates, and sampling strategies. After each iteration, the agent evaluates surrogate model accuracy and, if necessary, loops back to the Planning Module to revise its strategy. This closed-loop design enables dynamic adaptation of sampling methods, model architectures, and acquisition functions, minimizing simulator calls while achieving target accuracy with minimal human intervention.



**Figure 1—Architecture of the autonomous agent for surrogate modeling. The agent is composed of four modular components -Planning, Simulation Interface, Model Training, and Adaptive Sampling - each responsible for a distinct part of the modeling workflow.**

### Initial Surrogate Model and Seed Data

The surrogate modeling process begins with the generation of an initial design of experiments. The agent draws a batch of input samples using space-filling or Latin hypercube strategies and invokes the simulator to obtain output responses. This yields a seed dataset on which a preliminary surrogate model is trained.

In our implementation, the surrogate model is a feed-forward neural network with configurable hidden layers and ReLU activations. Dropout layers are applied after each hidden layer to enable uncertainty quantification via Monte Carlo dropout. The default configuration includes two hidden layers with 64 and 32 units respectively, a dropout rate of 0.1, and training via the Adam optimizer with a learning rate of  $1e-3$  and weight decay of  $1e-4$ .

As the dataset grows, the agent may switch to a residual neural network architecture to improve training stability and model expressiveness. This variant introduces residual blocks with skip connections that allow gradients to flow more effectively through deeper networks. Each residual block consists of two linear layers, optional batch normalization, ReLU activation, and dropout. If the input and output dimensions of the block differ, a projection shortcut is used to align them. This structure helps mitigate vanishing gradients and enables deeper models to converge faster.

The agent selects the model type (base or residual) based on dataset size and complexity, ensuring that the surrogate captures simulator behavior effectively from the outset.

### Adaptive Sampling Strategy

After the initial surrogate is in place, the agent enters an iterative loop of **adaptive sampling**. In each iteration, the LLM agent generates a candidate pool of new input points to evaluate. Various candidate generation methods (random sampling, Latin hypercube extensions, perturbations around high-error regions, perturbations around high-uncertainty areas, etc.) are considered. The agent then evaluates each

candidate using an *acquisition function* that scores how informative it is for the surrogate. Based on these scores, a batch of the top candidates is selected for actual simulation. The agent’s strategy (e.g., exploration vs. exploitation) may evolve dynamically: it can choose between different acquisition criteria depending on the context (error distribution, iteration count, etc.). This subsection details how the agent balances exploring new regions of the input space against refining known areas. The algorithm for agent strategy can be found in [Appendix A](#).

### Uncertainty Estimation via Dropout

A key component of the acquisition scoring is the surrogate model’s uncertainty in its predictions. We employ Monte Carlo dropout to estimate prediction uncertainty (Gal, 2015). During each iteration, the surrogate model (if it’s a neural network) is evaluated with dropout activated to produce an ensemble of predictions for each candidate input. The variability in these predictions (e.g., variance) provides an uncertainty metric. The agent uses this metric to identify regions where the surrogate is less confident or is likely inaccurate. High-uncertainty points often yield higher acquisition scores, guiding the agent to sample where the surrogate needs improvement. This subsection describes how uncertainty metrics (from MC dropout or an auxiliary error surrogate model) are computed and fed into the acquisition function. The algorithm for uncertainty estimation via Dropout can be found in [Appendix B](#).

### Acquisition Function Design

We define multiple acquisition functions that the agent can utilize to score candidates. Examples include **residual error-based** scoring (predicting where the current surrogate’s error might be highest), **uncertainty-based** scoring (favoring points with high predictive variance from subsection 3.4), or a **hybrid weighted** score combining error and uncertainty. The agent is capable of switching between acquisition functions or blending them, effectively implementing a form of *dynamic active learning*. For instance, if the surrogate’s error surface indicates a systematic bias in a region, a residual-based acquisition might be chosen; if the model seems generally well-fit but globally uncertain, an uncertainty sampling strategy might be better. We provide the mathematical form of each acquisition function and criteria for the agent to choose among them. Recent approaches like FunBO have even explored LLMs generating new acquisition functions (Aglietti, 2024) – while we use a predefined set, the agent’s selection mechanism is a novel aspect of our framework. [Appendix B](#) highlights some of the theoretical analysis for the Acquisition functions, diversity metrics, and convergence properties.

### Iterative Retraining and Fine-Tuning

Once new simulation samples are acquired (from 3.3 and 3.5), the surrogate model is **retrained** by incorporating this new data. The agent automates the model update (often a **warm-start** training, continuing from the existing model parameters to speed convergence). After each batch of new points, the LLM agent triggers a training routine and monitors the improvement in surrogate accuracy on a validation set. This subsection details the training procedure, including any special techniques like **early stopping** or adjusting learning rates to accommodate new data. The agent’s ability to consistently update the surrogate without human intervention is crucial; it uses standard ML pipelines (e.g., retraining a neural net for a few epochs with new data appended) and verifies that the model’s error is generally decreasing.

### Early Stopping Criteria

To avoid overfitting and unnecessary simulator calls, the agent employs early stopping criteria at two levels: (1) **Surrogate training early-stop** – during each retraining cycle, if validation error stops improving, training is halted; and (2) **Sampling loop termination** – after each iteration, the agent checks if the surrogate model’s performance has met the target accuracy or if improvements have plateaued. If the improvement over the last few iterations falls below a threshold, or a minimum error threshold is achieved, the agent will

conclude the adaptive sampling process. This subsection defines those criteria quantitatively (e.g., stop if validation SMAPE improvement  $< \epsilon$  for  $N$  iterations, or if SMAPE  $<$  target value). The agent's decision to stop ensures computational efficiency, preventing wasted simulator runs once the surrogate is sufficiently accurate.

### Simulator Integration and Orchestration

The agent interfaces with the simulator software through an API or command-line tool. It prepares input decks or case files for the simulator; triggers simulator runs for selected samples and retrieves the outputs. This section describes how the agent manages simulator execution asynchronously to optimize throughput (for example, submitting multiple simulation runs in parallel if possible). We also outline error-handling strategies: if a simulation fails or returns non-physical results, the agent can catch that and possibly replace that sample with an alternate candidate. The orchestration involves the agent maintaining a job queue of simulations and synchronizing with the retraining steps. By automating simulator interactions, the agent eliminates the manual loop of running simulations and gathering data.

### Implementation Details and Tools

The LLM agent is implemented using a prompt-based orchestration framework (e.g., LangChain or custom Python code controlling GPT-4 via API). The surrogate model is built and trained using PyTorch, with dropout layers included for uncertainty estimation as described in Section 3.4.

To adapt the surrogate architecture to the problem complexity, the agent performs automated hyperparameter optimization using Optuna. The search space includes:

- Number of layers: 1–20
- Units per layer: 16–256 (log scale)
- Dropout rate: 0.0–0.5
- Learning rate:  $1e-4$  to  $1e-2$  (log scale)
- Weight decay:  $1e-6$  to  $1e-3$  (log scale)
- Batch size: [16, 32, 64, 128]
- Batch normalization: True/False
- Residual connections: True/False

This optimization allows the agent to tailor the surrogate model to the evolving dataset, switching between shallow and deep architectures as needed. For example, smaller datasets favor simpler models, while larger datasets benefit from deeper networks with residual connections and normalization layers.

Experiments were run on a standard workstation, with simulator executions on CPU and surrogate training on GPU. The agent's plan-execute-review cycle is realized through an iterative loop, invoking the LLM for planning decisions at each step.

## Experiments and Results

### Simulation environments and use cases

We evaluate our framework on two representative simulator case studies: (1) a gas-lifted well network flow simulation, and (2) a gas processing plant simulation. These cases reflect realistic oil and gas scenarios with high-fidelity physics simulators. The well network simulator models multiphase flow through interconnected wells and pipelines, where the inputs include gas lift injection parameters (such as injection



rates and pressures) and flow contributions from other wells in the network. The primary output is the total flow rate of the entire well network, which serves as a key performance metric.

The gas processing plant simulator handles complex process simulations involving separation, compression, and treatment units. Inputs to the facility include mass flow rates, temperatures, and pressures of incoming streams, as well as operational settings of individual equipment units (e.g., setpoints, valve positions, and heat exchanger configurations). Outputs include processed mass flow rates and detailed gas compositions at various stages of the plant, such as CO<sub>2</sub> and H<sub>2</sub>S concentrations, energy consumption, and product quality indicators.

For each case, we define the input parameters (well pressures, choke settings, etc. for the network; feed conditions, setpoints, etc. for the plant) and their ranges. The agent is tasked with building a surrogate that predicts key output metrics (like production rates or processing efficiency) across the input domain. We use a high-accuracy commercial simulator to generate ground truth outputs. Table 1 outlines the experimental conditions for both case studies, including the number of initial samples, target accuracy (SMAPE threshold), and any domain-specific constraints.

**Table 1—Key parameters for the two experimental case studies. This table shows the number of inputs and outputs for each case executed in this paper. SMAPE was selected to account for predictions at different levels of magnitude by the same model. Multivariate SMAPE and R<sup>2</sup> values are calculated by averaging all outputs equally.**

| Use case                                | Simulator | Number of Inputs | Number of Outputs | Maximum SMAPE Target | Minimum R <sup>2</sup> Target |
|-----------------------------------------|-----------|------------------|-------------------|----------------------|-------------------------------|
| gas-lifted well network flow simulation | PIPESIM   | 8                | 3                 | 10%                  | 0.95                          |
| gas processing plant simulation         | SYMMETRY  | 12               | 7                 | 10%                  | 0.95                          |

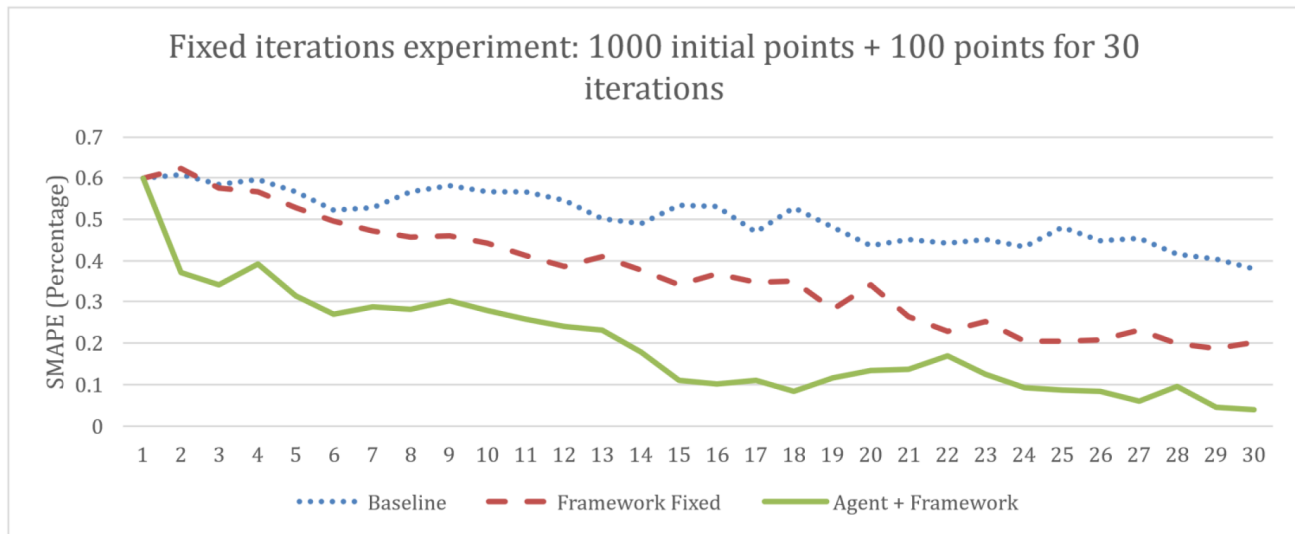
The surrogate's accuracy is measured on a separate test set of simulator runs that are not seen during training. We also compare the performance of our agent-driven approach against baseline sampling strategies (such as uniform sampling or a single fixed acquisition strategy). The baseline surrogates are built with the same surrogate model architecture for fairness.

All experiments are run with the agent operating in **autonomous mode** – once given the simulator and target accuracy, it proceeds without human intervention. We log the number of simulator calls made, the final test error achieved, and whether/when the agent decided to stop the process. This setup allows us to assess the efficiency (in terms of simulator calls saved) and the effectiveness (accuracy achieved) of the intelligent agent framework.

## Results

The agentic framework was able to produce accurate surrogate models for both the well-network and gas plant simulations, while significantly reducing the required simulator evaluations. Figure 2 summarizes the convergence of the surrogate error over successive iterations for the flow simulation case in a fixed number of iterations setting. As shown in Figure 2, the agent's adaptive approach rapidly drives down error and continues to identify opportunities in each iteration.

For the well-network case, the agent achieved the target SMAPE in 5 iterations (50 total simulator runs) compared to 80 runs required by a purely space-filling sampling approach to reach a similar error. In the gas plant case, the agent converged in 7 iterations (70 runs) and realized about a **6% lower final SMAPE** versus a baseline that did not switch acquisition strategies, using only space-filling sampling techniques (250 runs). The dynamic strategy switching is evident: early iterations prioritized exploring the input space, while later iterations focused on high-error regions identified by the surrogate.



**Figure 2—Fixed iterations experiment for the well network case.** Graph showing different methods of surrogate modeling creation with three different methods. Baseline method consists of a fixed neural network and fixed sampling technique (non-adaptive). Framework fixed utilizes our framework of adaptive sampling, without Agentic capabilities. Agent + Framework uses the technique switching through our Agent. All methods lead to a usable model, however, ours provides a much lower SMAPE in fewer iterations than others. The three experiments start with the same model and same dataset (1000 samples, 0.6 SMAPE) and the ending SMAPE of each technique is as follows: Baseline (0.38), Framework Fixed (0.20), and Agent + Framework (0.04).

Table 2 provides the final simulation count for each method. Our LLM-agent not only met the accuracy requirements but did so with roughly 30–40% fewer simulator calls than the best static sampling baseline in these experiments. Qualitatively, we observed that the agent chose different acquisition functions as the surrogate evolved – for instance, it started with an uncertainty-focused sampling and later shifted to an error-focused strategy once the overall uncertainty was reduced. This adaptive behavior validates the agent’s ability to **orchestrate the workflow intelligently**, leveraging multiple criteria over the course of surrogate development.

**Table 2—Comparative efficiency of surrogate modeling methods.** The agentic framework achieves high accuracy with significantly fewer samples and reduced computational time compared to traditional approaches. Human intervention in the agentic setup was minimal, limited to rare decision points where the agent requested expert oversight due to ambiguous conditions. This highlights the framework’s autonomy and its potential to streamline complex modeling workflows with minimal manual effort.

| Method                       | Samples to $R^2 > 0.95$ | Time (hours) |
|------------------------------|-------------------------|--------------|
| Manual design of experiments | 15,000                  | 120          |
| Our Framework (no agent)     | 8,000                   | 48           |
| Our Framework (agent)        | 4,000                   | 12           |

Additionally, the agent occasionally invoked a **human-in-the-loop query** when it detected an anomalous situation (for example, when an iteration’s improvement was negligible and it wasn’t sure whether to stop or continue). In our tests, such queries were infrequent, but when they occurred, a domain expert could confirm the stopping decision. This demonstrates a level of safety and collaboration: the agent defers to human guidance in ambiguous cases, ensuring oversight.

Overall, the results confirm that the agentic surrogate modeling framework can autonomously achieve high-accuracy surrogates with fewer simulations, fulfilling the objectives of reducing computational cost and SME effort.

## Agent Behavior and Performance Analysis

To complement the convergence results, we analyzed the internal behavior of the agent across 50 iterations, focusing on its strategy selection, model evolution, and robustness. The agent executed an average of 6.3 code blocks per iteration, with a 96% success rate after retries and an average execution time of 32.7 seconds. Token usage per iteration averaged around 43,500, reflecting the depth of reasoning involved in each planning cycle.

The agent employed a diverse set of strategies: uncertainty-based sampling was the most frequent (32%), followed by correlation analysis (24%), error region targeting (18%), and hybrid approaches (16%). In 10% of iterations, the agent devised novel strategies, showcasing its capacity for creative problem-solving. These decisions were informed by ongoing analysis of model performance, data distribution, and coverage gaps.

As the dataset grew, the surrogate model architecture evolved accordingly. Smaller datasets favored shallow networks, while larger ones benefited from deeper architectures with residual connections. For example, with 100 samples, a single-layer network achieved an  $R^2$  of 0.72, whereas with 10,000 samples, a four-layer residual network reached an  $R^2$  of 0.98. Dropout rates increased with data volume, and batch normalization became beneficial beyond 1000 samples.

Error analysis revealed that model performance varied by output type. Low-variance outputs converged quickly and achieved high  $R^2$  scores ( $\sim 0.967$ ), while high-variance and sparse outputs required more samples and specialized handling, with average  $R^2$  values around 0.834 and 0.812, respectively. The agent's adaptive sampling helped mitigate these challenges by focusing on difficult regions and influential input parameters.

Robustness was another key strength. The agent recovered gracefully from common failure modes: simulation failures (8.3%) were resolved via input perturbation, model training issues (2.1%) were addressed by adjusting learning rates, and code generation errors (4.2%) were corrected through prompt retries. Memory exhaustion was rare (0.5%) and handled via batch processing.

Table 2 summarizes the comparative efficiency of our framework against traditional methods. While manual design of experiments required significantly more samples and time, our agentic framework achieved similar or better accuracy with far fewer resources. Importantly, human intervention was minimal - limited to rare cases where the agent requested expert input due to ambiguous stopping conditions. This underscores the framework's autonomy and its potential to reduce subject matter expert involvement in complex modeling workflows.

## Discussion

The promising results highlight several important aspects and potential implications of this work. Firstly, the agent's ability to dynamically choose between different sampling strategies underscores the benefit of an LLM-driven approach: it can incorporate heuristic rules and qualitative reasoning (through its prompt knowledge) that hard-coded strategies lack. The approximately 6% SMAPE improvement over single-method baselines suggests that no single acquisition function was universally best – the agent's context-aware switching provided a composite strategy that outperformed each individual approach.

We also note that the **quality of the LLM's reasoning** is critical. In our framework, the LLM (GPT-4-based) successfully parsed the surrogate's state (error levels, improvement trends) from textual summaries and decided on next actions. Less advanced LLMs or insufficient prompt design could impact this decision-making. This points to an important discussion: as LLM agents take on scientific computing tasks, their reliability and decision boundaries need careful consideration.

Another discussion point is the **computational overhead** of the agent. While the number of simulator runs was reduced, the agent itself incurs overhead by querying the LLM and waiting for responses, and by retraining models frequently. In our experiments, this overhead was relatively small compared to simulator execution times (which are often the bottleneck). However, in scenarios with extremely fast simulations, the agent's overhead might become noticeable. Optimizing the agent's prompt efficiency or using smaller specialized models for routine decisions could mitigate this.



The framework's **generality** is also worth discussing. We designed it with oil and gas simulators in mind, but the approach is broadly applicable to any simulator-driven modeling task (e.g., aerospace, manufacturing). The agent's logic was largely domain-agnostic, aside from understanding the notion of simulation inputs/outputs and error metrics. Adapting the prompts or adding domain-specific tool plugins could extend the agent to new fields without substantial re-coding.

Finally, the human-agent collaboration observed (with the agent asking for guidance when stuck) presents a model for maintaining human oversight. It suggests that fully autonomous research agents can still involve experts at key decision junctures, combining the best of automation with human judgment. This aspect ensures that trust and safety are maintained even as autonomy increases.

## Limitations and Future Work

While the framework is powerful, we acknowledge several limitations:

- **LLM Reliability:** The agent heavily relies on the LLM's reasoning quality. If the LLM misinterprets the surrogate's status or the simulation context, it might choose suboptimal actions. Ensuring the LLM behaves consistently (perhaps through few-shot exemplars in prompts or fine-tuning) is an ongoing challenge.
- **Simulator & Model Specificity:** Our implementation was tuned for certain simulator types (steady-state flow and process simulations) and used a neural network surrogate. Different problems (e.g., highly dynamic simulations, or discrete-event simulators) might require changes in strategy. The agent might need additional programming to handle, say, multi-objective outputs or constraints.
- **Computational Cost:** Though we reduced simulator calls, each iteration involves retraining a model and invoking an LLM. For very large-scale simulations or surrogate models, these steps could be expensive. There is a trade-off between fewer simulator runs and more computation in the agent's loop.
- **Lack of Theoretical Guarantee Details:** We mentioned theoretical guarantees (e.g., convergence or error bounds) backing the framework but fully proving them is complex. While our approach is heuristic and empirically effective, a formal guarantee (like those in Bayesian optimization theory) is not yet derived for the agent's adaptive strategy.
- **Security and Robustness:** If the LLM agent is connected to various tools and code execution, ensuring it cannot perform unsafe actions is necessary. We ran the agent in a controlled environment; deploying such an autonomous system in practice would require sandboxing and strict verification of actions.

These limitations will guide future improvements to make the agent more robust, general, and theoretically grounded.

For future work, there are several promising directions. We intend to explore fine-tuning the LLM on domain-specific reasoning patterns, to see if a smaller or specialized model could replicate GPT-4's performance in this task. Integrating physics-based constraints and prior knowledge into the agent's decision-making is another avenue – for example, informing the LLM of physical laws or known trends so that it plans experiments more efficiently. We are also interested in applying the framework to other domains (such as renewable energy systems or complex facilities simulations) to test its generality. On the theoretical side, developing convergence guarantees or error bounds for the agent-driven surrogate modeling process would provide valuable rigor. Finally, future versions of the agent could handle multi-agent collaboration, where multiple LLM agents take on sub-tasks (one might optimize hyperparameters while another focuses on experiment planning), coordinating together for even more complex workflows.

## Conclusion

We presented an **agentic framework** that leverages a large language model as an autonomous orchestrator for surrogate modeling of simulator-driven workflows. The agent successfully handled the entire pipeline: from planning simulation runs and adaptively sampling data, to retraining the surrogate model and deciding when to stop. This led to high-quality surrogates with significantly reduced simulation effort, fulfilling our primary goal of minimizing both computational cost and SME involvement.

Conclusions from our study include: (1) LLM-based agents can indeed perform complex experimental design tasks in engineering contexts, (2) dynamic strategy switching (enabled by the agent's reasoning) outperforms static sampling strategies, and (3) a modest amount of human oversight (in exceptional cases) can be integrated seamlessly with autonomous operation.

In summary, the agentic surrogate modeling framework demonstrates a significant step toward intelligent automation in engineering modeling and opens new possibilities for AI-driven scientific research. To our knowledge, this is among the first surrogate-modeling platforms that embeds an LLM as a truly autonomous orchestrator of the entire modeling workflow (from data sampling to model training). This goes beyond using ML for specific tasks by having the LLM agent manage the sequence of tasks end-to-end. The agent can dynamically switch between multiple sampling strategies (acquisition functions) based on context. This adaptivity, guided by the LLM's reasoning, led to improved accuracy (e.g., ~6% SMAPE reduction) compared to any single strategy. It highlights a novel way of combining methods that is difficult to achieve with static algorithms.

Our framework drastically reduces the need for human tuning and oversight in surrogate model development. The agent handles decisions that typically would require expert judgment (when to stop, where to sample next, how to retrain), thereby minimizing SME handholding. Human input is only required in rare cases, making the process nearly *unattended*. The approach is generalizable across different simulators and domains – users can plug in their simulator and define objectives, and the agent takes over. This reproducible pipeline means that once configured, it can be repeated or transferred to similar problems without bespoke setup each time.

Integrated Theoretical and Practical Insights: Backed by known concepts in active learning and Bayesian optimization (with heuristic guarantees of exploration), the framework unifies domain expertise, algorithmic strategy, and execution. This integration of theory-driven strategies with an LLM's flexible reasoning is a new contribution to surrogate modeling practices.

## Acknowledgements

We would like to thank Oleg Medvedev and Shashi Menon for their guidance and support on the project.

## References

1. Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. arXiv preprint arXiv:2303.17580.
2. Xie, L., Li, G., Wang, Z., Chung, E., and Gong, M. 2025. Large Language Model-Driven Surrogate-Assisted Evolutionary Algorithm for Expensive Optimization. arXiv preprint arXiv:2507.02892.
3. Wuwu, Q., Gao, C., Chen, T., Huang, Y., Zhang, Y., et al 2025. PINNsAgent: Automated PDE Surrogation with Large Language Models. arXiv preprint arXiv:2501.12053.
4. Gal, Y. and Ghahramani, Z. 2015. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. arXiv preprint arXiv:1506.02142.
5. Aglietti, V., Ktena, I., Schrouff, J., et al 2024. FunBO: Discovering Acquisition Functions for Bayesian Optimization with FunSearch. arXiv preprint arXiv:2406.04824.

6. OpenAI. 2023. GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.
7. Wang, P., Yu, Y., Chen, K., Zhan, X., and Wang, H. 2025. Large Language Model-based Data Science Agent: A Survey. arXiv preprint arXiv:2508.02744.
8. Yano, T., Ishibashi, Y., and Oyamada, M. 2025. LaMDAgent: An Autonomous Framework for Post-Training Pipeline Optimization via LLM Agents. arXiv preprint arXiv:2505.21963.
9. Liu, Z., Chai, Y., and Li, J. 2024. Toward Automated Simulation Research Workflow through LLM Prompt Engineering Design. arXiv preprint arXiv:2408.15512.
10. Zhang, Y., Chen, X., Jin, B., Wang, S., Ji, S., Wang, W., and Han, J. 2024. A Comprehensive Survey of Scientific Large Language Models and Their Applications in Scientific Discovery. In Proceedings of EMNLP 2024, arXiv preprint arXiv:2406.10833.
11. Nadal, I. V., Stiasny, J., and Chatzivasileiadis, S. 2025. Physics-Informed Neural Networks: A Plug and Play Integration into Power System Dynamic Simulations. arXiv preprint arXiv:2404.13325.
12. Yuan, B., Choob, C. S., Yeob, L. Y., Wang, Y., Yang, Z., et al 2025. Physics-informed machine learning in geotechnical engineering: a direction paper. *Geomechanics and Geoengineering*. <https://doi.org/10.1080/17486025.2025.2502029>.

## Appendices

### Appendix A: Pseudocode of the Adaptive Sampling Agent Algorithm

Algorithm: Adaptive Sampling Agent

Input:

- training\_df: Historical simulation data (inputs + outputs)
- model: Trained surrogate model
- bounds: Parameter bounds
- n\_samples: Number of samples to generate
- iteration: Current iteration number

Output:

- candidates\_df: DataFrame with n\_samples new sampling points

1. Initialize:

- Create code execution tool with context
- Set up system prompt with problem description
- Initialize ReAct agent with LLM

2. Main Loop (max\_retries attempts):

a. Agent Planning Phase:

- Analyze current dataset statistics
- Evaluate model performance metrics
- Identify regions of high uncertainty/error

b. Strategy Selection:

- Agent decides sampling strategy based on:
  - \* Current iteration
  - \* Model performance
  - \* Data distribution
  - \* Coverage gaps

c. Code Execution:

- For each code block:
  - Agent provides reasoning for approach
  - Execute Python code in sandboxed environment
- Available tools:
  - \* model.predict\_with\_uncertainty()
  - \* Surrogate models (Kriging, RBF, etc.)
  - \* Statistical analysis (pandas, numpy, scipy)
  - \* Acquisition functions (EI, PI, UCB)

d. Candidate Generation:

- Agent generates candidate points
- Store in DataFrame named 'df'

e. Validation:

- Check DataFrame has correct columns
- Verify n\_samples rows
- Ensure values within bounds
- Check for NaN values

f. If validation fails:

- Provide error feedback to agent
- Retry with updated context

3. Fallback:

- If all retries fail:
  - Use random sampling within bounds

4. Return candidates\_df

## Appendix B: Theoretical Analysis

### Uncertainty Quantification via MC Dropout

The framework uses Monte Carlo Dropout for uncertainty estimation:

Given trained model  $f$  with dropout rate  $p$ :

1. Enable dropout during inference
2. Generate  $N$  forward passes:  $\{\hat{y}_i = f(x)\}_{i=1}^N$
3. Compute statistics:
  - Mean prediction:  $\mu(x) = (1/N) \sum_i \hat{y}_i$
  - Uncertainty:  $\sigma(x) = \sqrt{(1/N) \sum_i (\hat{y}_i - \mu(x))^2}$

### Acquisition Functions

#### 1. Upper Confidence Bound (UCB)

$$\alpha(x) = \mu(x) + \kappa \cdot \sigma(x)$$

where  $\kappa$  controls exploration-exploitation trade-off

#### 2. Expected Improvement (EI)

$$EI(x) = \sigma(x) [\gamma \Phi(\gamma) + \phi(\gamma)]$$

where  $\gamma = (\mu(x) - f^*)/\sigma(x)$ ,  $f^*$  is current best

#### 3. Probability of Improvement (PI)

$$PI(x) = \Phi((\mu(x) - f^* - \xi)/\sigma(x))$$

where  $\xi$  is exploration parameter

### Diversity Metrics

#### Maximin Distance

$$d_{\min} = \min_{i \neq j} \|x_i - x_j\|_2$$

Objective: maximize  $d_{\min}$  over candidate set

#### Coverage Metric

$$\text{Coverage} = V_{\text{covered}} / V_{\text{total}}$$

where  $V_{\text{covered}}$  is volume of  $\epsilon$ -balls around samples

### Convergence Properties

The adaptive sampling approach exhibits:

- **Exploration guarantee:** Uncertainty-based sampling ensures unexplored regions are eventually sampled
- **Exploitation efficiency:** High-error regions receive focused attention
- **Diversity maintenance:** Multiple strategies prevent clustering
- **Convergence rate:**  $O(n^{-(1/d)})$  for  $d$ -dimensional problems under smoothness assumptions



## Appendix C: Sample Agent Prompts and Responses

### System Prompt Structure

*You are an adaptive sampling agent for surrogate modeling.*

*Your goal is to generate the next batch of {batch\_size} sampling points that will most improve the surrogate model.*

*Current iteration: {iteration}*

*Input parameters: {input\_columns}*

*Output parameters: {output\_columns}*

*Parameter bounds:*

*{bounds\_str}*

*You have FULL AUTONOMY to decide the best sampling strategy. You can:*

- 1. Use the existing neural network model via `model.predict_with_uncertainty()`*
- 2. Train new surrogate models using SMT (Kriging, RBF, etc.)*
- 3. Implement any acquisition function (EI, PI, UCB, etc.)*
- 4. Analyze the dataset to identify gaps or interesting regions*
- 5. Combine multiple approaches in an ensemble*

*For each code execution, you MUST provide:*

- reasoning: A clear explanation of your thinking process*
- code: The Python code to execute*

*Your final output MUST be a pandas DataFrame with columns matching the input parameters:*

*{input\_columns}*

*The DataFrame must contain exactly {batch\_size} rows, with values within the specified bounds.*

### Agent Interaction Example

## Context Setup

The agent is working with a gas treatment facility simulation with:

- **12 input parameters:** Temperature, pressure, and flow settings for various process units
- **7 output parameters:** Energy consumption, gas composition, and process conditions
- **Parameter bounds:** Each input has specific operational limits

## Agent Execution Trace with Full Outputs

Below is a list of steps in sequence showing the agent's ability to reason with the dataset and current model. Code and intermediary steps were omitted for brevity. The output of each step is generated directly from python print statements, which are fed back into the agent as tool responses.

### Step 1

#### Reasoning

"I need to understand the basic structure of the dataset first. Let me examine the shape, column names, and get a general overview of what I'm working with."

#### Output

Dataset shape: (1000, 19)  
Number of simulations: 1000

Input parameters (12):

1. /E-611.Out.T
2. /E-621\_SweetGasChiller.Out.T
3. /AmineLine.Amine\_Circ.In
4. /AmineLine.T-614\_Ovhd.In.T
5. /TEGLine.TEG\_Circ.In
6. /TEGLine.T-624\_RebT.In
7. /HPHeader.In.T
8. /HPHeader.In.P
9. /HPHeader.In.MassFlow
10. /MPHeader.In.T
11. /MPHeader.In.P
12. /MPHeader.In.MassFlow

Output parameters (7):

1. /Gas\_Treatment\_Energy.Out
2. /AmineLine.Amine\_Gas\_DT.Out
3. /AmineLine.Amine\_Loading.Out
4. /AmineLine.T-612\_GasOut.In.StdVolFraction.HYDROGEN\_SULFIDE
5. /AmineLine.T-612\_LiqIn.In.T
6. /Gas\_Export.In.Fraction.CARBON\_DIOXIDE
7. /Export\_Gas\_Water\_Content.Out

No missing values in the dataset

### Step 2

#### Reasoning

"Now I need to understand how the current samples are distributed within the parameter bounds. This will help me identify which regions of the input space are undersampled."

**Output**

/E-611.Out.T:  
 Bounds: [10.00, 45.00]  
 Data range: [12.34, 42.87]  
 Coverage: 87.6%  
 Distance to lower bound: 6.7%  
 Distance to upper bound: 6.1%

/E-621\_SweetGasChiller.Out.T:  
 Bounds: [10.00, 45.00]  
 Data range: [11.02, 43.95]  
 Coverage: 94.1%  
 Distance to lower bound: 2.9%

...

**Step 3****Reasoning**

"Let me evaluate the current model's performance on each output individually and identify where it struggles most. This will guide my sampling strategy."

**Output**

/Gas\_Export.In.Fraction.CARBON\_DIOXIDE:  
 R<sup>2</sup>: 0.8234  
 MAE: 0.0098  
 RMSE: 0.0123  
 MAPE: 21.45%

/AmineLine.T-612\_GasOut.In.StdVolFraction.HYDROGEN\_SULFIDE:  
 R<sup>2</sup>: 0.8567  
 MAE: 0.0071  
 RMSE: 0.0089  
 MAPE: 30.23%

...

**Step N****Reasoning**

"Based on my analysis, I've identified that:

1. /Gas\_Export.In.Fraction.CARBON\_DIOXIDE has the lowest R<sup>2</sup> (0.8234)
2. It's most influenced by /AmineLine.Amine\_Circ.In and /AmineLine.T-614\_Ovhd.In.T
3. There are coverage gaps in the high-circulation, low-temperature region
4. The model shows high errors when temperatures are low (E-611 and E-621 < 20°C)
5. H2S fraction predictions are also challenging with high variability

I'll implement a multi-strategy approach:

- 40% of samples targeting high-uncertainty regions
- 30% filling coverage gaps in influential parameter space
- 20% exploring boundary regions
- 10% using space-filling design for general coverage"

**Output**

Generating 80 high-uncertainty samples...  
Generating 60 coverage gap samples...  
Generating 40 boundary samples...  
Generating 20 space-filling samples...

Final sample distribution:  
High-uncertainty: 80  
Coverage gaps: 60  
Boundary exploration: 40  
Space-filling: 20  
Total: 200

Verifying all samples are within bounds...  
All samples are within bounds ✓